="brand">Альфа / Поток

Актуальная оценка готовности проекта

Кредитный кабинет предпринимателя и рабочее место андеррайтера. Оценка выполнена по фактическому коду, тестам и локальному smoke-тесту.

90%
готовность контролируемого MVP-стенда

56%
готовность к production -контуру

Дата: 25 октября 2026

Ветка: `main`

Локальный commit: `8b93b2d` поверх `origin/main eb06f5`d

Состояние: оценка включает незакоммиченные изменения текущего рабочего дерева; push не выполнялся.

class="page">

1. Итог

Вердикт: MVP готов к контролируемой демонстрации и функциональной приёмке.> Клиентский и банковский пути связаны реальным API, серверными расчётами, аудитом, WebSocket-обновлениями и polling fallback. Критические пользовательские замечания текущего прохода закрыты.

Это ещё
не банковский production.
SQLite, отсутствие миграций и transactional outbox, публичный demo reset, локальные секреты, отсутствие rate limiting и frontend-тестов не позволяют считать систему промышленно готовой.

Как посчитаны 90%
Итог — взвешенная

оценка готовности именно контролируемого MVP-стенда. Дробный результат 89% округлён до 90%. Production readiness вынесена отдельно и не маскируется функциональной полнотой демо.

Направление	Вес	Оценка	Основание

Сквозной продуктовый сценарий20%97%Подбор → заявка → решение → сведения/оффер → подписание → моделируемая выдача. Frontend и UX15%93%Grouped results, статусы, причины, skeleton, уведомления, история и даты оценки. Нет browser automation. Backend/API/workflow20%95%Роли, state machine, аудит, DTO, повторная оценка и realtime после всех workflow-мутаций. Decisioning и объяснимость15%92%Детерминированные правила, constraints/reasons/actions, stress, версии и snapshots. Политика демонстрационная. Тестирование и QA10%94%101 backend/расчётный тест, lint/build и live smoke. Нет frontend component/E2E/load tests. Документация и сдача5%90%README, единый CONTRACTS, OpenAPI и актуальный отчёт. Deployment/наблюдаемость8%68%Dockerfiles, Compose, healthcheck есть; локально Docker CLI недоступен, production deployment не перепроверен. Security и production data7%51%Роли и HttpOnly-cookie есть; остаются secrets, Secure cookie, CORS/Origin, rate limit, PostgreSQL и миграции.

2. Что реализовано

Кли

ент

- Авторизация и восстановление HttpOnly-cookie сессии.
- П

рофиль компании, финансы и синтетические документы.

- Подбор

по цели, сумме, сроку и приоритету.

- Полный grouped response: requested, other terms, reduced amount, requires action, unavailable.
- С

водка исходного запроса, продуктовые проверки, structured reasons/constraints и область сравнения.

- Я

вные изменения срока и суммы без подмены исходного запроса.

- Создание заявки, дополнительные сведения, принятие/отклонение

, подписание и демовыдача.

Банк

- Очередь заявок с

клиентским поиском и фильтрами.

- Карточка компании, запроса, checks, score

и stress snapshot.

- Approve, alternative, need_info, reject и отзыв актив

ного оффера.

- Обязательное обоснование для bounded review score 50–69.
- Комментарий банка и клиента отображается в истории

после повторного чтения.

- Точная дата исходной оценки отделена от текущ

его профиля и оффера.

Realtime и UX

- Аутентифицированный WebSocket и фильтрация получателей.
- `application.updated` после create, additional info, accept/decline, sign, disburse и bank decision.
- Reconnect, полная синхронизация и polling раз в 15 секунд как fallback.
- Ручная кнопка «Обновить данные» удалена.li>
- Об

щий доступный PageSkeleton с reduced-motion.>

- Увед

омления закрываются outside click и Escape с возвратом focus.

Расчё

ты и аудит

- Decimal-аннуитет, coverage и ограничения продукта.
- Rule-based score 0–100 с factors и версионированием.
- Точные assessment: eligible, alternative, need_info, manual_review, blocked, unavailable.
- Stress-сценарии 0/10/20/30% и объяснение cash-flow дефицита.
- RecommendationAudit и audit решения банка со snapshots/h

ashes.

- UTC

timestamps и воспроизводимые серверные расчёты.

Принципиальная граница

> **В проекте нет обученной AI/ML-модели.**
> Скоринг — прозрачный rule-based baseline. Ranking — детерминированная сортировка. Assistant — шаблонный демонстрационный endpoint. Stress — анализ чувствительности, а не прогноз дефолта.

> Эти элементы нельзя выдавать за обученный AI, PD-модель или автоматическое банковское одобрение.

3. Фактическая валидация

Проверка	Фактический результат	Статус
`.venv/Scripts/python.exe -m`		

pytest>>101 тест собран и пройден за 16.37 с. Workflow, API, scoring, financing, ranking, request explanations, review policy, stress и timestamps.PASS `npm run lint` 0 warnings, 0 errors; 31 файл, 116 правил.PASS `npm run build` TypeScript и Vite build успешны; 688 модулей.PASS HTTP health `/api/v1/health` вернул healthy database connection.PASS >Live WebSocket через frontend proxyСоздание заявки вернуло `review`; банк получил `application.updated` с тем же application ID и статусом.PASS Demo reset после smokeSeed восстановлен после созданной smoke-заявки.<td class="status-ok">PASS `git diff --check` Whitespace errors не обнаружены.PASS Docker ComposeНе запускался: локальный Docker CLI отсутствует (`docker: command not found`).td>НЕ ПРОВЕРЕНОtr>

Неблокирующие предупреждения

- Production bundle

JavaScript: около 605.76 KB (gzip около 181.09 KB); Vite предупреждает о chunk больше 500 KB.

Известный `npm audit`: 2 moderate issue в React Router 6; автоматический fix

требует breaking upgrade на v7.

Frontend пока не имеет component/integration/E2E test

runner; lint/build не заменяют браузерные тесты.

Zed Python diagnostics не используют проектную `.venv`; runtime pytest является фактической проверкой импортов.

4. Что осталось до production

<tr>

Приоритет	Риск / gap	Что сделать
P0	SQLite	

, `create_all`, нет Alembic; конкурентная семантика не подтверждена.PostgreSQL, миграции, rollback-план, интеграционные и concurrency tests. P0Demo secret, Secure cookie выключен, публичные `/demo/reset`, mocks и assistant.Secrets manager, HTTPS/WSS, закрыть demo endpoints, CORS/Origin из env, rate limiting. P0In-memory WebSocket registry, нет broker/outbox/replay.Redis/NATS/Kafka или иной pub-sub, transactional outbox, delivery/replay policy. P0Backend API напрямую допускает создание заявки с blocked/need_info/unavailable assessment.Утвердить продуктовую политику и enforce её на сервере, не только в UI. P1Нет idempotency keys и системной optimistic version.Идемпотентность create/decision/transitions, version column и конфликтные тесты. P1Нет реальной загрузки новых документов и обновления финансов в need_info.Object storage/AV scan/metadata, source freshness, новый recalculation workflow. P1OpenAPI errors/security описаны не полностью; list→detail остаётся N+1.Единый error schema, security schemes, rich/batch bank endpoint и server-side filters. P1Нет observability и нагрузочной проверки.Correlation ID, structured logs, metrics/traces, alerts, HTTP/WS/load/failure tests. P2Frontend bundle больше 500 KB и нет browser automation.Route/code splitting; Vitest/RTL и Playwright для критического пути и accessibility.

5. План

по ролям

Frontend

1. Добавить Vitest/React Testing Library для контекстов и компонентов.
2. Добавить Playwright:

две роли, realtime, outside click/Escape, responsive и a11y smoke.

- Разбить bundle по маршрутам; измерить LCP/INP

на стенде.

- Добавить визу

альные regression snapshots ключевых grouped states.

Backend / Platform

1. PostgreSQL + Alembic + проверка блокировок и последовательности application ID.
2. Idempotency/optimistic concurrency и outbox/pub-sub.
3. Production auth

/security hardening и закрытие demo endpoints.

- Наблюдаемость, backup

/restore, retention и нагрузочные тесты.

Аналитик / продукт

1. Утвердить каталог, ставки, лимиты, score/coverage thresholds и manual review

.

- Решить допустимость blocked

/unavailable application и правила alternative.

- Утвердить владельца disburse

, срок действия оффера и need_info workflow.

- Зафиксировать data dictionary: asOf, период, источник, owner, freshness SLA.>

ML / Data Science

1. Не обучать модель на synthetic labels и не называть baseline AI.
2.

Сначала определить target, horizon, observation window и valid outcomes. >Подготовить leakage-safe split, calibration, fairness и monitoring policy.

- Только после данных

сравнить ML с rule-based champion baseline.

6. Сценарий сдачи MVP

1. **Подготовка.strong> Запустить backend/frontend, проверить**

health, выполнить demo reset. Открыть клиента и банк в разных browser profiles из-за общей cookie.

Прозрачный подбор. Войти клиентом, открыть финансирование, изменить цель/сумму/срок. Показать requestSummary, requestChecks и отдельные группы.

Альтернатива. Показать, что срок/сумма отличаются явно, а исходные поля не подменяются. Продемонстрировать reasons, constraints, comparison labels и свёрнутый stress.

Создание заявки. Выбрать проходящий или bounded-review вариант и отправить заявку. На банковском экране заявка должна появиться по WebSocket без ручного refresh.

Need info. Банк запрашивает сведения с комментарием. Клиент получает notification, отправляет ответ; комментарии обеих сторон остаются в истории после reload.

Решение. Банк одобряет либо предлагает альтернативу с обоснованием. У клиента сразу обновляются статус и уведомление.

Завершение. Клиент принимает оффер, подтверждает подписание и моделируемую выдачу. На каждом шаге банк получает application.updated.

Границы. Проговорить, что score rule-based, stress не является прогнозом, assistant шаблонный, а выдача и внешние проверки моделируются.

> **Критерий приёмки стенда**
> Весь сценарий выполняется без ручной синхронизации, статусы обеих ролей сходятся, причины и комментарии не теряются, а UI нигде не называет предварительную оценку окончательным одобрением.

Решение о

выпуске
Для hackathon/demo/MVP: GO после короткого ручного браузерного прогона на целевом стенде.

Для пилота с реальными финансовыми или персональными данными: NO-GO до закрытия P0 по данным, безопасности, миграциям и гарантированной доставке.

Отчёт сформирован по локальному состоянию рабочего дерева 25.10.2026. Push, VPS deployment и Docker Compose validation в этот проход не выполнялись.